2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00047

AsyncDIMM: Achieving Asynchronous Execution in DIMM-Based Near-Memory Processing

Liyan Chen, Dongxu Lyu, Jianfei Jiang, Qin Wang, Zhigang Mao, Naifeng Jing*
Department of Micro/Nano Electronics, Shanghai Jiao Tong University
{liyan.chen, sjtuj}@sjtu.edu.cn

***Abstract*—DIMM-based near-memory processing (NMP) architectures address the "memory wall" problem by incorporating near-memory accelerators (NMAs) into main memory devices for high memory bandwidth and low energy consumption. However, critical challenges prevent efficient asynchronous execution between host and NMAs in DIMM-NMP architectures. Memory controllers (MCs) distributed at the host side and the NMA side issue memory accesses independently without synchronization on memory states, which may lead to memory bus contention and DRAM errors. Therefore, most existing DIMM-NMP designs adopt synchronous execution to prevent concurrent memory accesses. However, this intervention wastes either the host or the NMA computation capability.**

In this work, we propose AsyncDIMM, a novel DIMM-NMP design with efficient asynchronous execution based on existing memory buses. It enables single access mode (host or NMA), concurrent access mode, and a seamless switch between them. First, we propose the *offload-schedule-return* mechanism with explicit and implicit synchronization to ensure memory access correctness for all memory modes. Second, to further improve bandwidth utilization and decrease access latency, we introduce optimized timing constraints for offloading, a locality-aware switch-recovery method for scheduling, and adaptive batch with timing-division multiplexing notification for returning. Finally, we present a detailed design with limited hardware modifications to conventional host and NMA MCs, which is extensively validated on the FPGA. Comprehensive experiments demonstrate that AsyncDIMM outperforms four NMP baselines by $1.19\times$-$1.92\times$, enabling efficient asynchronous execution with up to $2.25\times$ bandwidth utilization uplift and 47% access latency reduction.

I. Introduction

DIMM (Dual-Inline Memory Module)-based near-memory processing (NMP) that integrates accelerators on the buffer chip has emerged as an attractive computing paradigm to address the "memory wall" challenge between host and main memory [1]–[3]. Compared with conventional memory systems which leverage shared memory buses to access ranks (e.g., two ranks per DIMM) with fixed channel bandwidth, the aggregated bandwidth of DIMM-NMP scales with the number of DIMMs. As reported, DIMM-NMP has been applied to domains such as recommendation systems [4]–[9], machine learning [10], [11], genomics analysis [12], [13], graph processing [14], etc., with significant speedup and power savings.

Ideally, in NMP systems, compute-intensive workloads or portions execute on the host, while the memory-intensive workloads can be offloaded to near-memory accelerators (NMAs), thus leveraging both high computation ability of the host and high bandwidth of the NMAs. However, this potential gain has not been effectively exploited in existing DIMM-NMP designs [15], [16], due to limitations inherent in the DIMM-NMP architectures, primarily stemming from the memory controller (MC).

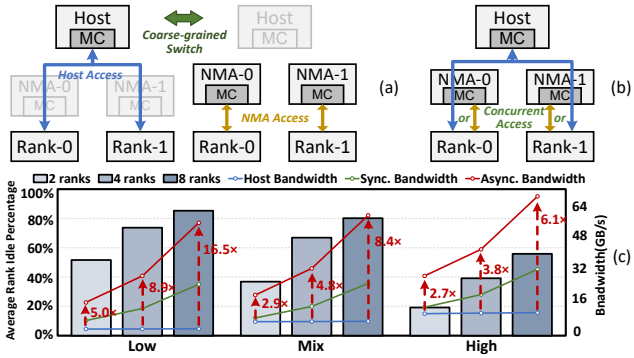


Fig. 1. (a) An example of synchronous execution. (b) An example of asynchronous execution. (c) Rank idle time and bandwidth analysis.

Unlike conventional architectures that leverage a shared host MC, DIMM-NMP architectures feature MCs not only on the host side but also on the NMA side. In this case, concurrent memory accesses independently issued by host and NMA MCs could lead to protocol violations, such as memory bus contention and DRAM physical errors [16]. For instance, host MC and NMA MC may access different rows in the same bank simultaneously, which is not supported by the shared internal buses and not allowed in DRAM arrays [17].

Consequently, most existing DIMM-NMP designs adopt synchronous execution [1], [18]–[20], where host and NMA execution are switched in a coarse-grained manner, as demonstrated in Figure 1(a). Concurrent memory access is prohibited for correctness and MCs perform synchronization during switch. However, this intervention underutilizes either the host or the NMA computation capability. As shown in Figure 1(c), for a range of workloads from SPEC CPU2006 [21] and SPEC CPU2017 [22], executed on an 8-core host CPU (methodology in Section VI), the rank idle periods exceeding 50 cycles could be 37.0%, 62.1%, 75.6% with 2, 4 and 8 ranks per channel, respectively. This bandwidth waste arises because within a channel, the host MC can only access a single rank at a time, leaving the other ranks idle during host execution.

Based on this observation, we identify that asynchronous execution, as shown in Figure 1(b), can better utilize the wasted bandwidth, thus achieving higher performance gain than synchronous execution in DIMM-NMP architectures. A

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00047

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

few recent works have made efforts to enable asynchronous execution but still struggle under unacceptable inefficiency. For instance, UPMEM [2] and NDA [23] segregate DIMMs accessed by host and NMAs, which degrades both the computing parallelism of NMP and the bandwidth available to the host, as fewer DIMMs can be accessed by each. Chopim [16] enables concurrent access by host MC tracking all NMA accesses. However, it imposes restrictions on the NMA design and its workloads, requiring them to have a predetermined access pattern and latency. Meanwhile, packet-based communication [24], [25] leveraging the link or network-on-chip (NoC) can be used for asynchronous execution in DIMM-NMP. However, it necessitates drastic changes to DIMM interfaces and also leads to higher access latency [26]–[28].

In this paper, we propose AsyncDIMM, a novel DIMM-NMP design with efficient asynchronous execution based on existing memory buses. We introduce explicit and implicit synchronization between host MC and NMA MC to facilitate a seamless memory mode switch. We further propose the *offload-schedule-return* (OSR) mechanism for concurrent memory access. Specifically, on the host MC side, we design new memory commands to *offload* memory access with optimized timing constraints. On the NMA MC side, we implement the switch-recovery (SR) method to *schedule* host offloaded access and NMA local access, which maximizes row locality while ensuring the correctness of memory states. To *return* read data from the NMA MC, we propose the adaptive batch with timing-division multiplexing (TDM) interrupt, which improves bandwidth utilization and reduces access latency. In summary, our main contributions are as follows:

- We deeply review existing methods for synchronous execution and analyze their limitations in respect of hardware overhead, generality and overall performance.
- We propose AsyncDIMM, a novel NMP design to achieve efficient asynchronous execution in DIMM-NMP architectures. We present detailed designs of mechanism implementation, hardware architecture, etc., which are also extensively validated on the FPGA.
- Comprehensive experiments show that AsyncDIMM outperforms four NMP baselines by $1.19\times$-$1.92\times$, enabling efficient asynchronous execution with up to $2.25\times$ bandwidth utilization uplift and 47% access latency reduction.

II. Background and Motivation

A. DIMM-NMP Architecture

As illustrated in Figure 2, the main memory system consists of independent channels. In each channel, the DIMMs are connected to the MC through a command/address (C/A) bus and a DQ (data) bus. A DIMM is composed of ranks, which are further organized as bank groups (BG) and banks. Each bank consists of multiple rows and a row buffer. Depending on the memory levels where NMAs are integrated, DIMM-NMP can be categorized as rank-level (on buffer chip) [4], [6], [10] , BG-level [8], bank-level [2], [29] and even cross-level [8], [9] architectures. Typically, an NMA is composed of a compute

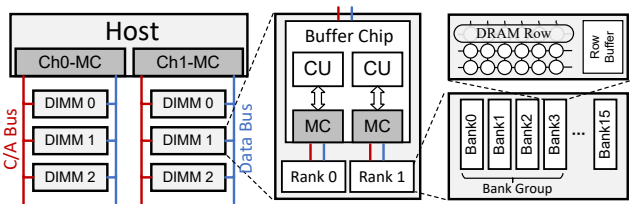


Fig. 2. Main memory system and rank-level NMP architecture.

unit (CU, such as Cores [2], ASIC [30], [31], processing elements (PEs) [16], [32], etc.) for computation and a local MC [2], [31] for memory access. Based on the execution flows, NMP architectures can be classified as coarse-grained [18]–[20], [33] or fine-grained [9], [34]–[37]. With coarse-grained flow, NMAs start execution and leverage the local MC to exclusively control their memory after host offloads kernels. In fine-grained execution flow, NMAs continuously accept instructions from the host and decode them into corresponding micro-kernels or DDR commands without ability to actively use the local MC. *This paper focuses on the more general rank-level architecture with a coarse-grained execution flow.*

B. Memory Controller

The MC schedules DDR commands [40] for memory access, such as activate (ACT), precharge (PRE), read (RD), write (WR), etc. When a memory bank receives an ACT (with row address), this bank would be open and fetch the selected row data into the row buffer. Subsequently, a WR or RD (with column address) can transfer data to or from selected columns of the row buffer. If the next request goes to a different row, a PRE should be issued to close this bank, writing data in the row buffer back to the corresponding row. For simplicity, we call commands that change the bank state as bank state change (BSC) commands, such as ACT and PRE.

The timing constraints for a DDR command and between commands under specific memory states are explicitly defined and have to be strictly obeyed [17], [41], [42]. An MC is generally designed with finite state machines (FSMs) to record the memory states and generate DDR commands for memory requests, schedulers to calculate timing constraints to be obeyed for pending commands, and an arbiter between banks or ranks to arbitrate which command and when to issue.

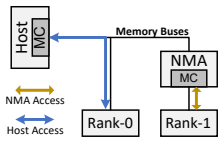
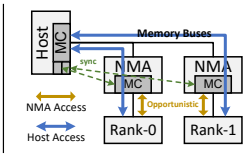
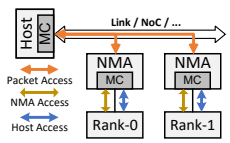
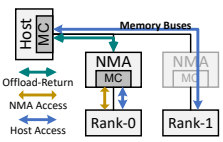
In conventional systems, the MC is integrated per channel at the host side, shared by all DIMMs or ranks [43]–[45]. In DIMM-NMP systems, MCs are distributed at both the host side and NMA sides, whose designs are quite different. Host MC needs to record the states of the whole channel, and NMA MC only records states of its corresponding rank. Meanwhile, the host MC implements various scheduling methods to exploit row locality [46], [47]. The NMA MC tends to use simple FIFOs to complete memory requests sequentially due to area and power considerations [4], [31].

C. Asynchronous Execution Methods

There are mainly three methods related to asynchronous execution in DIMM-NMP, namely shielding, opportunistic,

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISONS OF HOST-NMA EXECUTION METHODS

Methods	Shielding Access [2], [23]	Opportunistic Access [16]	Packet-based Access [38], [39]	**Our Work**
Illustration				
Hardware Modifications	-	NMA CU Host MC, NMA MC	Memory Buses Host MC, NMA MC	**Host MC, NMA MC**
Generality Considerations	Limited Memory Mode	Limited CU & Workloads	Limited DIMM Type	**General Purpose**
Performance	Parallelism Restricted	Schedule Restricted	Schedule Restricted	**High BW & Low Latency**

and packet-based access methods. We compare them in detail and list the limitations they are faced with as follows.

Shielding Access. The shielding access method [2], [23] segregates memory devices accessed by host and NMAs. As shown in the second column of Table I, during NMA execution, the host can only access the conventional ranks (or ranks with inactive NMAs). UPMEM [2] exemplifies this method, which, despite being a bank-level NMP design, is also applicable to rank-level NMP architectures. At the memory device side, a multiplexer (MUX) arbitrates between host and NMA accesses. At the host side, the software development kit (SDK) [48] keeps track of the MUX state to guarantee exclusive ownership. Before NMA execution, the host sets the MUX. Upon completion, the NMA notifies the host and updates the MUX state. This method requires no hardware modifications (albeit with necessary software support) to existing host MC, facilitating its adoption in other NMP designs.

However, this method still precludes host access to NMP-enabled memory during NMA execution, imposing constraints on workload data layout, MC address mapping and dynamic memory allocation strategies. Furthermore, it limits the performance of both host and NMP due to the reduced number of available DIMMs for each side, given the system's finite DIMM slots. For instance, only two conventional DIMMs per socket are employed for host execution in UPMEM configuration [15], significantly diminishing the system's effective storage capacity and memory bandwidth.

Opportunistic Access. Chopim [16] implements a real-time synchronization mechanism, enabling the host MC to track all NMA local memory accesses. As shown in the third column of Table I, the NMA MC can opportunistically issue local memory accesses without concern for potential simultaneous access from host MC. To facilitate this, the NMA workloads should have predetermined compute and memory patterns, whose latency and access can be tracked by the host. When the NMA MC issues access, the host MC also synchronizes with it. Since the tracking is unidirectional (only host to NMA), Chopim prioritizes host access to prevent host starvation, while issuing NMA access with a predefined probability.

However, the opportunistic access method is only compatible with restricted workloads. For instance, workloads with dynamic execution states, such as Breadth-First Search (BFS) whose loop length is not fixed, or with dynamic memory accesses, such as Sparse Matrix-Vector Multiplication (SpMV) where accessed data is dynamic, are not supported. Furthermore, the NMA CU and NMA MC should also be designed to be latency-determined for host tracking. Worse still, the scheduling between host and NMA accesses lacks row locality exploitation, in which the predefined probability can only allocate bandwidth between host and NMAs in an empirical manner. Consequently, the row buffer hit rate of host accesses decreases sharply with concurrent NMA accesses (28%-64% degradation in our evaluation), which results in increased access latency ($1.36\times$-$2.03\times$) for both sides.

Packet-Based Access. The packet-based access method in other memory devices, such as Hybrid Memory Cube (HMC) [24], [25], HMC-NMP [38], [49], [50] and Compute Express Link [39], holds potential for enabling asynchronous execution in DIMM-NMP. As shown in the fourth column of Table I, the host communicates with NMA MC via packets instead of DDR commands, which can eliminate protocol violations between the host MC (replaced by a packet controller such as HMC controller) and NMA MC. For instance, in HMC architectures, each device connects with the host through a dedicated off-chip link, while vaults in the device are connected to the I/O interface via a crossbar (simplified NoC) [51]. The dedicated link and crossbar ensure no bus contention between HMC devices and between vaults during packet transfer, respectively.

However, there are mainly three limitations in applying it to DIMM-NMP. First, it leverages LxRX(P/N) and LxTX(P/N) as interface signal pins [51], which lead to substantial interface modifications on both sides [26]. Considering the signal integrity, the high-speed serial pins also compromise the plug-and-play feature [28]. Meanwhile, conventional DIMMs (without the local MC) are not compatible in this interface. Second, to prevent bus contention, the system requires either a dedicated link to each DIMM or a NoC between DIMM slots (e.g., on the PCB board), which incurs non-negligible

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

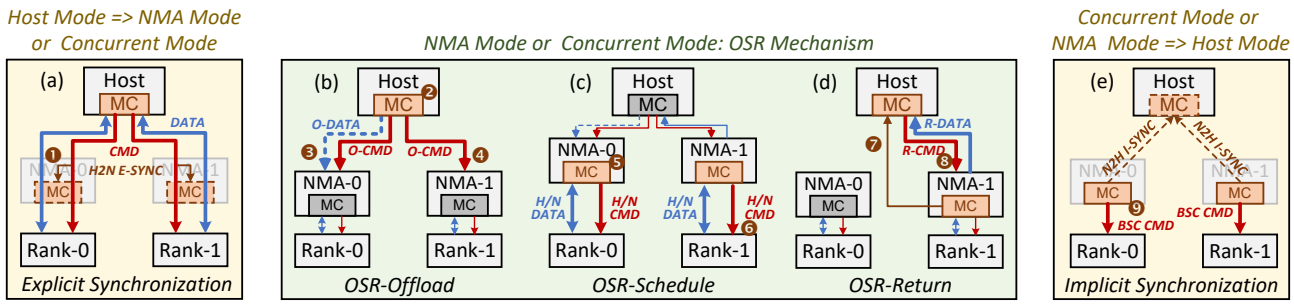


Fig. 3. OSR mechanism with explicit and implicit synchronization.

hardware cost. Third, the high performance of HMC is largely attributed to the sophisticated vault MC design [52]. In contrast, the MCs [4], [31] integrated on buffer chips are typically constrained in complexity due to area and power limitations. In our evaluation, the FIFO-based NMA MC can lead to 1.19×-2.27× latency compared with the host MC design, which is not efficient enough to handle the host accesses.

To sum up, existing DIMM-NMP designs encounter significant obstacles in achieving efficient asynchronous execution due to several key challenges: 1) costly hardware overhead or modifications; 2) limited generality concerning supported memory modes, NMA workloads, and memory devices; 3) restricted performance on bandwidth and latency. Therefore, there is an urgent need for efficient asynchronous execution in DIMM-NMP architectures with acceptable hardware overhead, broad compatibility, high bandwidth, and low latency.

III. Mechanism Overview

There are basically three memory modes in DIMM-NMP architectures: host, NMA and concurrent mode. To enable efficient asynchronous execution, AsyncDIMM not only considers these modes, but also the switch between them. First, in host mode, we employ explicit synchronization to update NMA MC, ensuring a seamless switch to the other two modes. Second, we propose the OSR mechanism for concurrent mode and NMA mode (the last column of Table I), which completes host memory access through offloading, scheduling (with NMA memory access) and returning based on existing memory buses. Third, implicit synchronization is introduced for switching back to host mode.

A. Explicit Synchronization

As shown in Figure 3(a), in host mode, DDR commands are issued from host MC to memory devices on the C/A bus, while data are transferred on the data bus. We further bypass the host-issued commands to the NMA MC simultaneously (❶). These commands update the recorded memory states in the NMA MC as if they were issued by the NMA MC itself. In this way, the NMA MC can always track the up-to-date memory states, called host-to-NMA (H2N) explicit synchronization, which ensures the seamless switch from host mode to NMA or concurrent mode. Meanwhile, maintaining the execution of NMA FSMs only introduces little power overhead compared to the memory access itself according to our evaluations.

B. Offload-Schedule-Return Mechanism

We propose OSR mechanism for NMA mode and concurrent mode, as shown in Figure 3(b)-(d). In NMA mode, host access is simply restricted from being issued. In concurrent mode, host access is offloaded from the host MC to the NMA MCs. Then the NMA MCs schedule host offloaded access and NMA local access. After completion, the data of offloaded read is returned to the host MC. Note that offloading, scheduling, and returning are not performed sequentially but simultaneously between memory accesses. Meanwhile, the OSR mechanism is compatible with conventional DDR memory access, supporting configurations where only some NMAs are working or where both NMP-enabled and non-NMP-enabled DIMMs coexist in the channel.

1) Offload Mechanism: Different from host mode where commands are issued directly to memory banks, the host MC offloads the commands to the NMA MCs instead in concurrent mode. We allow the host MC to generate and offload commands based on the FSMs even though they may record the incorrect memory states. In this way, the host accesses are already pre-scheduled before being offloaded (❷), with high row locality utilization between each other. As for the correctness of issuing incorrectly generated commands to DRAM arrays, it is ensured in the NMA MC scheduling.

We introduce four new memory commands, ACTO, RDO, WRO and PREO, as the offloading versions of ACT, RD, WR and PRE respectively. When host MC issues commands, the offloading versions are applied. For write access, the commands and write data are both offloaded (❸). For read access, only commands are offloaded and the read data would not be ready until NMA MC returns it (❹). We propose timing constraint optimizations for them to better exploit the existing memory buses. The host MC manages whether a command can be offloaded. Other memory operations, such as refresh (REF), are directly handled by NMA MC.

2) Schedule Mechanism: After being offloaded, host accesses are further scheduled in the FIFO-based NMA MC (❺). First, a FIFO is used for each bank to store offloaded commands. Since the commands are already scheduled by host MC, the row locality is inherently maintained in the FIFO, without introducing complex schedulers for area and power considerations. Second, the switch method is proposed to arbitrate between the FIFO of host commands and the FIFO

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

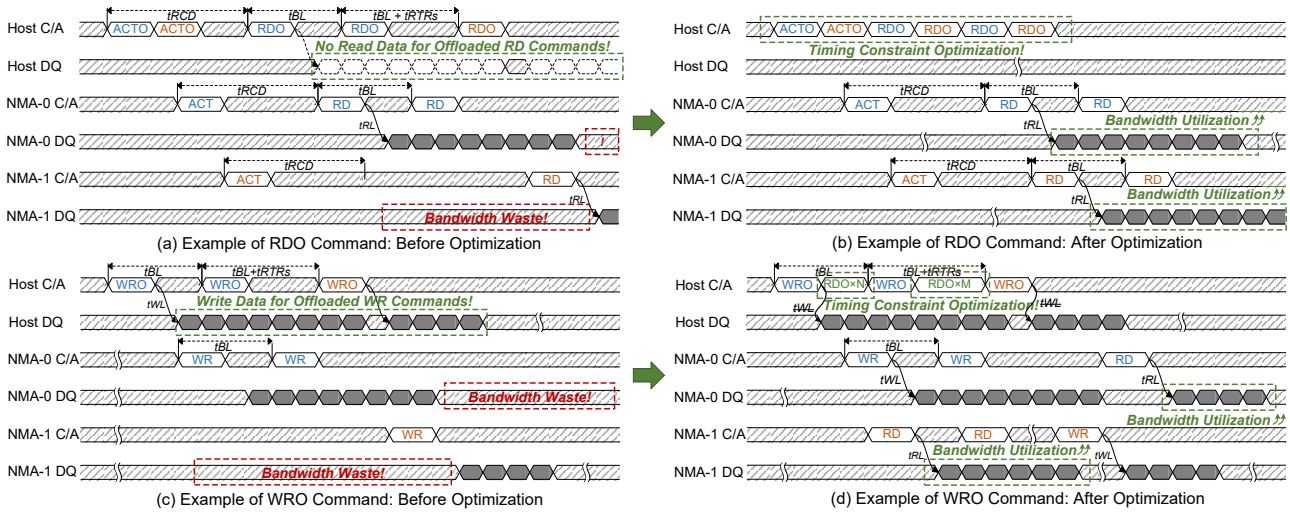


Fig. 4. Timing constraint optimization for offloading commands.

of NMA requests, maximizing row locality between memory accesses in two different forms. Last and most importantly, we propose the recovery method in NMA MC to issue DDR commands correctly (❻) since the host commands are generated based on incorrect memory states.

3) Return Mechanism: As mentioned above, when host MC offloads read accesses (e.g., ACTO+RDO commands) to NMA MC, the target data would not be read back after corresponding latency like normal reads. Moreover, after access completion, the data is only read into the NMA MC instead of host MC. Considering that the gating of the memory buses is controlled by the host MC, the DIMMs/ranks cannot transfer data to the host MC at will, as it would lead to bus contention and signal error. We further introduce a new memory command, Return (RT) command. Specifically, when the offloaded read access is completed in NMA MC, it should notify the host MC first (❼). Then the host MC sends RT to re-read the data from NMA MC instead of DRAM arrays (❽). To reduce the latency of the completed offloaded read in the NMA MC, we further propose the adaptive batch method with TDM interrupt.

C. Implicit Synchronization

For switching back to host mode, rather than helping the host MC to re-track the memory states, we change them to match the expectations of host MC FSMs, called NMA-to-host (N2H) implicit synchronization, as shown in Figure 3(e). It only performs at the end of concurrent and NMA modes, unlike the real-time explicit synchronization. Specifically, since the host offloaded commands are generated based on the FSMs, in turn, the last offloaded BSC command reflects the state recorded in the FSMs. For example, if the last BSC command is an ACT, the recorded bank state would be open, while an PRE corresponds to close. Based on it, we record the last-issued BSC command for each bank in NMA MC (❾). Before switching, the NMA MC checks if each bank state matches the expectation of the last host BSC command. If it does not (e.g., NMA accesses have changed it), the NMA MC reschedules corresponding commands to recover it. When all banks are ready for the next commands, the mode switches.

This synchronization seems time consuming if most banks need recovery. However, when NMA finishes execution, there are likely to be uncompleted host commands in NMA MC and the mode switch should be performed after all of them are completed. In this case, the synchronization can be done automatically by these commands or overlappingly with them. According to our evaluation, less than 2 banks need recovery in each mode switch on average, and more than half mode switches do not need recovery, which significantly saves latency and energy.

IV. Mechanism Detail and Optimization

We further build and optimize the OSR mechanism upon the existing memory buses. For offloading, we analyze and optimize the timing constraints of offload commands, which also mingle with existing DDR commands. For scheduling, we propose the SR method to arbitrate memory accesses with high row locality and maintain the correct memory states for DDR command issuing. For returning, we propose the adaptive batch with TDM interrupt to ensure the order of data return and to reduce its latency.

A. Offload: Optimized Timing Constraints

In conventional systems, timing constraints are mainly designed for DRAM arrays and bus occupancy. However, they are inefficient in the OSR mechanism because 1) the offloaded commands are not issued to banks; 2) the data may not occupy the memory buses. If they are directly used for offloading, severe memory access latency and parallelism degradation would be caused. Correspondingly, the optimization is based on these two features. We present examples with a configuration of two NMAs as illustrated below.

First, as shown in Figure 4(a), the timing constraints for continuous RD (RDO) commands maintain tBL for occupancy of DQ bus (tCCD_s or tCCD_l for BG and bank states not

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

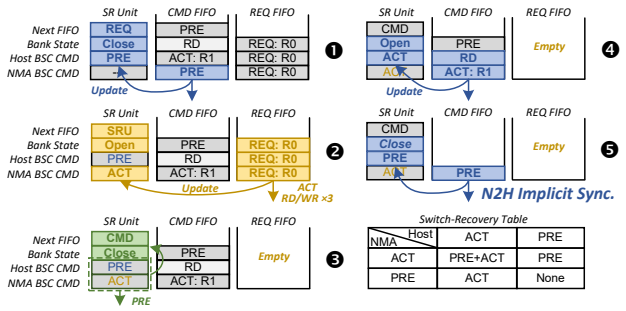


Fig. 5. An example of switch-recovery method. Rx for Row-x.

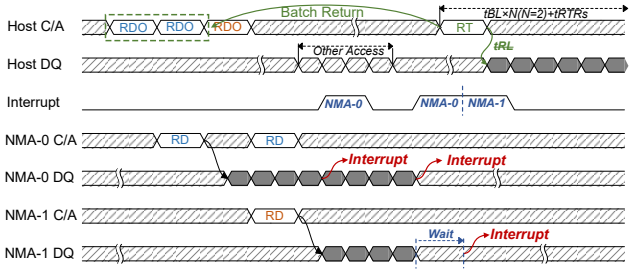


Fig. 6. Timing constraint design for RT.

shown) and tRTRs for switch between ranks. In this case, it causes one of the NMA MC (e.g., NMA-1) to stay idle, thus wasting bandwidth. Based on that the offloaded RD commands do not read data back from banks, we omit the corresponding timing constraints. Moreover, tRCD, which corresponds to opening a bank row, can also be omitted because offloaded ACT commands do not really change the bank states. As shown in Figure 4(b), the more efficient offload leads to higher aggregated bandwidth in NMA sides.

Second, as shown in Figure 4(c), the similar bandwidth waste could also be found for WR command offload. However, the timing constraints cannot be omitted in this case because write data occupies the data bus after issuing WR commands. To optimize this case, we insert read accesses between offloaded writes, if there are read requests (e.g., RDO commands) waiting for arbitration. It essentially omits tWTR (write to read latency) and tRTW (read to write latency) during offloading. Meanwhile, tWL for write latency can also be omitted. As shown in Figure 4(d), NMA MCs can achieve higher bandwidth utilization with more offloaded commands to schedule. Note that PREO and ACTO are required when the row switch occurs between WRO and RDO, which are not shown in the figure.

Finally, the offloaded commands with optimized timing constraints can also coexist with DDR commands with existing timing constraints. For instance, during normal WR to a non-NMP-enabled DIMM, the RDO could also be offloaded to NMP-enabled DIMMs in the same way like Figure 4(d). We identify that as the number of NMAs increases, the utilization of conventional timing constraints will cause more serious bandwidth waste. In this case, our optimized timing constraints for OSR mechanism are both efficient and scalable.

B. Schedule: Switch-Recovery Method

The row locality between offloaded commands can be inherently preserved in the FIFO structure. Consequently, host commands and NMA requests are segregated in the per-bank CMD FIFO and REQ FIFO in the NMA MC, respectively. However, arbitrarily switching between these FIFOs could significantly degrade bandwidth utilization since they usually share little row locality. Additionally, the offloaded commands, generated based on potentially incorrect memory states recorded in the host MC, cannot be directly issued by the NMA MC without risking conflicts with the actual bank states. To tackle these challenges, we introduce the SR method.

First, we propose to exploit the row locality of the current FIFO as much as possible and only perform the switch when necessary or with no overhead. Specifically, we restrict the switch to occur when one of the following conditions is met: 1) No more commands or requests in the FIFO; 2) The issue of a PRE command. Both conditions indicate that no more row locality could be exploited in the FIFO. The latter further ensures the switch with no overhead. Moreover, since MCs would issue PRE commands when row hits reach a pre-defined threshold, starvation in corner cases (e.g., continuous accesses to the same row) can be prevented in the method.

Second, we record the last BSC commands from both FIFOs to determine if the recovery is needed. Take an example of one bank shown in Fig 5. In Phase ❶, a PRE is issued from CMD FIFO, which updates the BSC CMD and the bank state recorded in the SR unit. In Phase ❷, the MC switches to REQ FIFO due to the issue of PRE. Then one ACT and three RD/WR are generated to complete all requests. In phase ❸, NMA MC switches and performs recovery. The SR unit checks the switch-recovery table to issue corresponding BSC command(s). After that, commands in CMD FIFO can be issued safely in phase ❹, and the N2H implicit synchronization is done automatically when the last PRE is issued in ❺.

As a consequence, when a REQ FIFO is selected, the NMA requests generate DDR commands based on the NMA MC FSMs. When a CMD FIFO is selected, the host offloaded commands can simply bypass the FSMs, which avoids modifying FSMs whose designs vary across different MCs.

C. Return: TDM Interrupt & Adaptive Batch

Similar to existing NMP designs [53], [54], we leverage the interrupt-based signal (e.g., one-bit alert_n signal) for NMA MC to notify host MC. However, this signal is shared by ranks like the C/A Bus and DQ bus. Once the alert_n signal is issued, host MC needs to poll all ranks to check the source. The polling latency (up to 50-80 cycles) is acceptable in acceleration design (e.g., notification of execution completion), but it is not acceptable for memory access. To tackle this challenge, we propose a simple but effective method that performs time-division multiplexing on the interrupt signal, which limits the effective cycle of each rank issuing the alert_n signal. Since the interface clocks of the host MC side and the DIMM side are synchronized during the calibration phase, host MC can

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

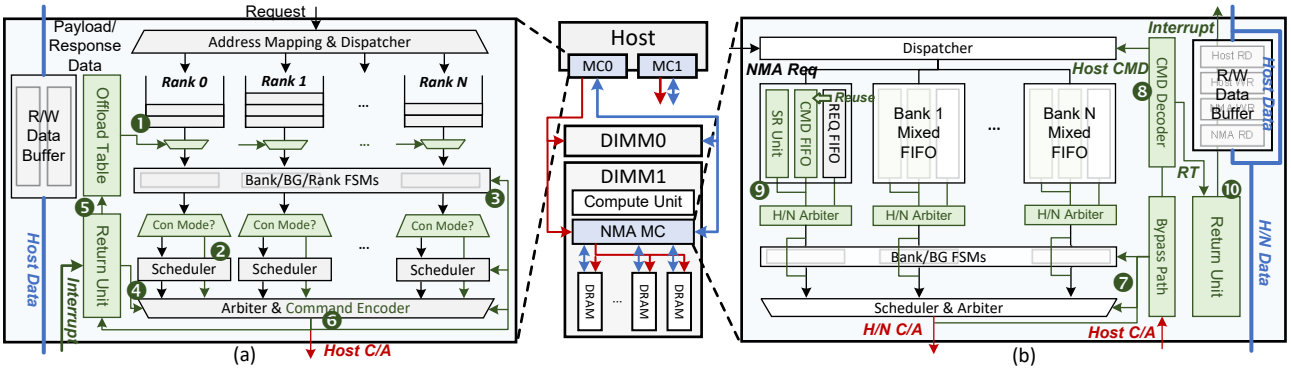


Fig. 7. Detailed architecture design of AsyncDIMM.

determine the source rank based on the cycle of the alert_n signal arrival. It only introduces latency averaging $N_{rank}/2$ cycles while eliminating the polling overhead completely.

One crucial problem is that the one-bit signal cannot notify the host MC of which offloaded memory accesses have been completed considering that they are out-of-order between banks. Therefore, we restrict the issued interrupts at the order of memory accesses being offloaded. When the interrupt for an older access has not been issued, the interrupt for more recent ones cannot be issued even though they have been completed. When the host MC asks the NMA MC to return data, it could have accepted more than one interrupt. Consequently, the read data can be returned collectively as a batch, whose size equals the number of interrupts from offloaded reads.

We further design the timing constraints of RT. Different from existing commands, its latency is not fixed but determined by the batch size. As shown in Figure 6, for the first RT command issued to NMA-0, the return data would occupy the data bus for tBL$\times$2, while tBL for the second RT command issued to NMA-1. We replace the tRL latency with NMA indexing time since the return data are not from memory banks but NMA buffers. Moreover, the RT can be regarded as one or multiple special RD commands without accessing memory banks. In this way, it can also coexist with existing DDR commands on memory buses without protocol violations.

V. Detailed Architecture Design

In this section, we dive into the more detailed architecture design of AsyncDIMM. In the host MC, we implement offload and return mechanisms tightly coupled with existing MC design with restricted modifications. Moreover, we ensure the offloaded commands can be successfully received without backpressure signals. In the NMA MC, explicit and implicit synchronizations are implemented. Meanwhile, we rebuild the request FIFOs to be configurable for commands which further reduces hardware overhead. Lastly, we consider other memory operations, host-NMA communication and priority issues.

A. Host MC Design

In the host MC, while retaining the mechanism to access conventional DIMMs, we enable access offloading and data returning for NMP-enabled DIMMs, as shown in Figure 7(a).

TABLE II
AsyncDIMM Command Truth Table

AsyncDIMM Command Truth Table for DDR4 (CS_n, ACT_n, etc., are not shown)											
Function	Abbreviation	RAS_n/ A16	CAS_n/ A15	WE_n/ A14	BG0-BG1	BA0-BA1	C2-C0	A12/ BC_n	A17, A13, A11	A10/ AP	A0-A9
Offload Activation	ACTO	RA			BG	BA	V: HHH	RA			
Offload Write	WRO	H	L	L	BG	BA	V: HHH	L	V	L	CA
Offload Read	RDO	H	L	H	BG	BA	V: HHH	L	V	L	CA
Offload Precharge	PREO	L	H	L	BG	BA	V: HHH	V	V	L	V
Return	RT	L	H	H	Batch Size			RFU			

H = Logic HIGH level; L = Logic LOW level;
V = Valid Signal; RA = Row Address; CA = Column Address;

1) Concurrent & NMA Mode : We implement concurrent mode and NMA mode with a uniform design. A one-bit register per rank is used to reflect if the mode is activated, which can also be set by the interrupt. We introduce Offload Table (OT) to record the number of commands per bank that have been offloaded. If it reaches the threshold, OT will prevent generating commands (❶). The threshold is set to the NMA FIFO size for concurrent mode and to 0 for NMA mode, respectively. Then the commands generated from request buffers bypass the schedulers (❷) to omit corresponding timing constraints. After being offloaded, the commands update FSMs, schedulers, and arbiters like normal issuing (❸). Finally, the Return Unit (RU) sends RT to get return data, with burst length of interrupt count from read accesses(❹). RT only needs to update the bus occupancy recorded in the arbiter. We set the issuing of RT as the highest priority over other DDR commands, to ensure low access latency.

2) Return Unit: The RU also records the information of offloaded memory accesses in order per rank, including physical address (32 or more bits), read or write (1 bit) and the number of commands (2 bits). For the last item, 2'b01 is for one RD/WR command, 2'b10 is ACT + RD/WR commands, and 2'b11 is for PRE + ACT + RD/WR commands respectively. These two-bit registers are used to update OT in real-time when new access is added or an old one is deleted (❺). For write access, it is deleted when receiving the corresponding interrupt. For read access, it waits for both interrupt and return data. When the address and data are valid, they can be inserted to read data buffer and respond to the higher memory level.

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

The size of RU is co-designed with the total size of all NMA CMD FIFOs. For example, a return table for 8 DIMMs or 16 ranks supporting 64 offloaded memory accesses is about 10KB, which is relatively small compared with the data buffer. Meanwhile, it can also be implemented in the same way as the data buffer, which is usually instantiated as embedded memory using SRAM in MC design (e.g., uMCTL2 [55]), and introduces acceptable hardware overhead.

3) Command Encodings: We encode the new commands with current C/A signals (❻). As shown in Table II, for offload commands (e.g., ACTO, RDO, etc.), we leverage the Valid Signal (High or Low, but not floating) in conventional ACT, RD, WR and PRE to distinguish them. For example, if the C2-C0 bits are all High, the NMA MC would regard them as offloaded commands and all Low for normal ones. For the RT command, we leverage the RFU (reserved for future use) command to encode it. According to the standard [17], while encoding cs_n: L, ACT_n: H, RAS_n/A16: L, CAS_n/A15: H and WE_n/A14: L, there are a number of configurable command signals for RFU operations. We use 7 bits (BG, BA and C2-C0) to encode the batch size and reserve the remaining bits for future use, such as priority, fixed batch size, etc.

B. NMA MC Design

The NMA is put on the DIMM buffer chip, as shown in Figure 7(b), such that the host MC could interface with NMAs through the standard memory channel.

1) Bypass Path: In host mode, host commands are issued directly to the memory banks through the bypass path (❼). They are further transferred to the FSMs, the schedulers and the arbiter to update memory states for explicit synchronization. Similarly, the write data or read data are transferred to or from memory banks directly, bypassing the data buffer.

2) Configurable FIFO: In concurrent mode, the offloaded commands are inserted into CMD FIFOs through the CMD decoder and the dispatcher (❽). AsyncDIMM does not introduce separate dedicated FIFOs but rebuilds the existing REQ FIFOs to be configurable. Specifically, in NMA mode, the NMA MC leverages the whole REQ FIFO to store NMA requests. In concurrent mode, half of the FIFO capacity is reused for host commands (❾). Since the width of commands is less than that of requests, we do not need to modify the FIFO width. The host-NMA arbiter is further introduced for each mixed FIFO unit to arbitrate between two FIFOs. Meanwhile, the SR unit records the BSC commands and checks if any command(s) should be generated to recover the bank state. The host commands are directly issued to the schedulers without checking bank states by FSMs. In this way, we do not modify the logic in FSMs to support more diverse inputs.

3) Return Unit: Similar to the host MC, the return unit is added to record the offloaded access states (❿), including information on address, read or write, data valid, interrupt issuing. It sends the interrupt to the host MC when the oldest access is completed. When receiving an RT from the CMD decoder, the NMA MC returns the batch data indexed by the physical address, whose size is determined in the batch size field of the RT command. The latency between the RT and its data is determined by NMA MC hardware parameters.

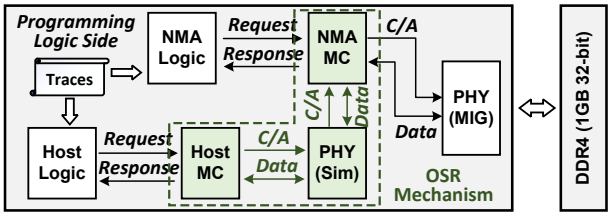


Fig. 8. AsyncDIMM OSR mechanism validation on FPGA.

C. Further Considerations

First, different from memory access operations, Refresh (REF), Self-Refresh Entry/Exit (SRE/SRX) and Mode Register Read/Write (MRR/MRW) [17] should be directly handled by NMA MCs in concurrent or NMA mode instead of being offloaded. Second, to support host checking NMA state, modifying parameter registers [31], [53] and other communications in real time, NMA MC can also accept host normal access whose address should be constrained to a pre-defined physical range [56]. The CMD decoder is responsible for handling it, which is not shown in the figure. Third, we focus on exploiting row locality for high bandwidth utilization with no consideration for priority between host and NMA accesses currently. We identify that it can be assigned with an additional RFU command. We believe that the priority design is an important trade-off for host-NMA latency and bandwidth.

VI. Evaluation

To comprehensively evaluate AsyncDIMM, we first assess the feasibility and efficiency of OSR mechanism using the FPGA. Then, we evaluate AsyncDIMM's performance against four NMP baselines through simulation.

A. FPGA Validation

1) Hardware Implementation: As presented in Figure 8, we validate AsyncDIMM hardware design of OSR mechanism using AMD Kintex UltraScale+ FPGA KCU-116. To support interaction between the host and the NMA, one standard memory interface, DDR PHY (sim), is instantiated. To access DDR4 devices on the board, DDR PHY (hard IP) is generated by memory interface generator (MIG [57]) IP's PHY-only configuration. Instead of generating an MIG MC, we implement a relatively simple host MC and an NMA MC based on an open-source MC [58], while further completing its timing constraint check, bank state tracking and scheduling strategies for FPGA design. The host and NMA logic generate memory accesses independently from pre-dumped traces.

2) FPGA Validation Scope: We leverage the FPGA to validate the OSR mechanism at the logic level, instead of electrical characteristics, signal integrity and so on. Based on the FPGA platform, we build an NMP system with one NMP-enabled rank. We conduct two evaluations to assess the feasibility and efficiency of OSR mechanism: (1) Concurrent access validation: Host and NMA generate memory requests

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

concurrently, with host verifying access correctness. (2) OSR latency analysis: Host generates memory requests and NMA remains inactive, where three approaches are compared: normal host access, OSR without timing constraint optimization, and optimized OSR.

3) Resource Utilization & Performance: We implement the hardware logic using Verilog HDL and synthesize it with Vivado 2019.1. We leverage the user interface clock generated in the DDR PHY (MIG) as the fabric clock. On the KCU-116 board, our design consumes of 5.49% LUT, 6.06% FF and 27.14% IO resources. On the one hand, the FPGA successfully completes concurrent memory accesses with OSR mechanism, which partially validates the feasibility of AsyncDIMM. On the other hand, compared with normal host access, non-optimized OSR introduces extra latency of averaging 34 cycles on the evaluated access traces. In contrast, optimized OSR shows negligible latency of less than 4 cycles on average, whose performance gain would even increase with more ranks. In the next section, we perform a simulation-based evaluation to facilitate the comparison with other methods.

B. Evaluation Methodology

1) Baselines: We compare AsyncDIMM with four baseline NMP methods: Chameleon [1], which adopts synchronous execution. UPMEM [2], which relies on the shielding access. Chopim [16], which adopts opportunistic access. HMC-Link [24], [49], which changes the DIMM interface to the packet-based interface. For UPMEM, we configure non-NMP-enabled and NMP-enabled ranks with the ratio of 1:3. The non-NMP-enabled ranks are distributed to channels for fair comparison (different from configurations [15] where they share a channel). The host only accesses NMP-enabled ranks in a coarse-grained manner. For Chopim, we follow its configuration and set the NMA write probability as $1/4$. For HMC-Link, we replace the memory buses with a link and connect DIMMs with a crossbar, which regards the NMA as the vault. The packet information includes both header and tail (e.g., 128 bits in an HMC-2 request packet [51]). We restrict the theoretical bandwidth of HMC-Link to be the same as other baselines.

2) Benchmarks: Our intention is to assess the efficiency of AsyncDIMM across different memory patterns, achieved through combinations of host and NMA workloads. In our current setup, these workloads execute separately on the host and NMAs, with shared data limited to the input and output of NMA workloads. We also acknowledge the significance of host-NMA collaboration workloads. Nonetheless, we recognize that the kernel partition, data layout, and execution flow are equally crucial as the execution mode for a certain workload, which could potentially form a dedicated study (e.g., host-NMA co-design [59], [60] for large language model).

For host workloads, we choose applications from the SPEC CPU2006 [21] and SPEC CPU2017 [22], [61] benchmarks. We use eight sets of multi-programming workloads, categorized based on *misses-per-kilo instructions* (MPKI) for various memory intensities [62]–[64]. Applications with LLC MPKI values larger than 10 will be identified as memory-intensive HIGH workloads. The remaining ones will be classified as LOW workloads. The MPKI values are chosen based on works [62], [65]. In the MIX workloads, four High and four Low applications are chosen at random for each set.

TABLE III
HOST AND NMA BENCHMARKS

Host Workloads	
ID	Applications
High-1	bwaves_r, lbm_r, fotonik3d_r, mcf_r, soplex, lbm, leslie3d, libquantum
High-2	mcf, omnetpp, bwaves, GemsFDTD, sphinx, soplex, milc, libquantum
Low-1	gcc_r, cactus_r, deepsjeng_r, leela_r, blender_r, imagick_r, nab_r, xalan_r
Low-2	deepsjeng_r, leela_r, xz_r, astar, bzip2, cactusADM, h264, perlbench
Mix-1	namd, gobmk, gamess, povray, mcf, omnetpp, bwaves, GemsFDTD
Mix-2	tonto, gromacs, hmmer, zeusmp, bwaves_r, lbm_r, fotonik3d_r, mcf_r
Mix-3	gcc_r, deepsjeng_r, leela_r, blender_r, lbm_r, fotonik3d_r, bwaves_r, mcf_r
Mix-4	nab_r, xalan_r, leela_r, xz_r, milc, omnetpp, GemsFDTD, libquantum
NMA Workloads	
ID	Applications
BK	COPY, ADD, SCALE, AXPY, TRIAD, GEMV
CK	SpMV, SpGEMM, FFT, TS, BFS

For NMA execution, we provide two types of workloads, basic kernels (BK) and complex kernels (CK). Following works [16], [66], [67], BK have determined execution latency and memory pattern, representing core computations in many algorithms, which serve for all baselines. In contrast, CK [15], [68], [69] have non-deterministic execution latency or memory patterns, which are used for all baselines excluding Chopim. The details of each workload set are presented in Table III.

3) Simulation: The performance of the five systems is evaluated with GEM5 simulator [70], [71] under X86 ISA for host modeling and DRAMsim3 [72] for both DRAM and NMA modeling. Each host workload is run on 8 cores with simulation fast-forwarded for one billion host instructions to get past the initial phases, then the caches warm up for another 100 million host instructions. Finally, the detailed simulations are run for one billion host instructions for each workload to collect the statistics. During detailed simulations, each application in NMA workloads is run exclusively in order, and the completion of all applications is considered to be one execution. Simulations for NMA workloads are launched the same number of times to ensure the same execution progress on the NMA side for fairness [16].

For power and area consumption of MCs, we synthesize them with Synopsys Design Compiler under 28nm CMOS technology to obtain their extra overhead compared with conventional MC implementation. For memory access, we utilize DRAMsim3 [72] to evaluate the energy of memory devices and Cacti-IO [73], [74] to evaluate the energy of off-chip IO. Since this paper does not focus on the NMA CU design, the evaluation does not compare its energy consumption.

4) Configuration: Table IV summarizes the system configurations. We set an 8-core OoO CPU running at 2.4GHz as the host CPU, while the NMA accelerators are at the memory device frequency of 1.6GHz. The host MC and NMA MC

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SIMULATOR DETAILS

Host CPU / CPU Baseline	
Processor	8 out-of-order cores, 2.4GHz, x86-64 ISA
L1I / LID / L2 / LLC	32KB,8 / 64KB,8 / 2MB,8 / 32MB,4
Host MC Scheduler	
Scheduler	FRFCFS, Open policy
REQ Queue	Size: 16, Structure: bank
NMA Accelerator	
Type	Configurable PEs / Trace-driven CPU
Parameters	Scratchpad Memory: 64KB, Frequency: 1.6GHz
NMA MC Scheduler	
Scheduler	CMD-REQ Mixed FIFO with SR Unit
CMD FIFO, REQ FIFO	Size: 8 for each, Structure: bank
Memory System: DDR4-3200	
Devices	2 Channels $\times$ 4 Ranks
Address Mapping	Row-Channel-Rank-Bank-BG-Column
DRAM Timing Parameters	CCD=4:RRD=4/8:RCD=22:RAS=52:RP=22:RC=74: CL=22:WL=16:CDLR=4/12:WR=24:CCDL=8:RTP=12

are modeled in detail with different schedule methods and the same per-bank FIFO size. Although a real system may implement a more sophisticated mapping scheme to improve performance, it is out of the scope of this study. The specifications of DDR4-3200 are based on an officially released paper by Micron [40]. Two kinds of CUs are employed: 1) configurable PE arrays for workload BK; 2) a CPU for workload CK. PEs are driven by bit-streams, which include kernel types, data address, data size, and other configuration parameters. The NMA CPU operates based on memory access traces, generated from GEM5 simulation on each application. In this paper, we focus on the diverse access patterns provided by both host and different CUs rather than on their specific designs. For a fair comparison, we maintain consistency across all configurations except for the MC.

C. Overall Performance

First, we leverage the speedup ratio to compare the performance on the host side, and use NMA internal bandwidth utilization to compare the performance on the NMA side.

1) Speedup: Figure 9 shows the performance comparison (main axis) of AsyncDIMM and four baselines, with performance normalized to Chameleon (the synchronous execution design). First, AsyncDIMM outperforms all baselines across all workloads, achieving average speedups of $1.92\times$, $1.25\times$, $1.26\times$, $1.19\times$, respectively. Chameleon and UPMEM fail to effectively exploit the parallelism between host and NMAs, thus underutilizing one of their compute ability. Chopim shows good speedup in the Low-0 and Low-1 workloads, but its gains diminish with increased host access intensity due to lack of scheduling for host and NMA accesses. Additionally, Chopim does not support concurrent memory access for NMA workload CK. HMC-Link, relying solely on NMA local MC with a FIFO-based schedule method, performs poorly in high-intensity host workloads (e.g., High-1 and High-2).

Second, AsyncDIMM and all baselines demonstrate better speedup with NMA workload BK over CK, despite similar NMA access intensity in the evaluation. This is due to BK's regular memory access pattern with better row locality, which generally perform better in the FIFO-based NMA MC. However, when the host access intensity is high (e.g., High-1 and High-2), the significant performance degradation occurs if host-NMA interaction is not effectively managed, as seen with Chopim and HMC-Link. In contrast, AsyncDIMM maintains stable host performance even under high access intensity, benefiting from its effective schedule method that maximizes row locality between host-NMA accesses.

2) NMA Bandwidth: Figure 9 also compares NMA internal bandwidth utilization (secondary axis). First, on the top of performing best on the host side, AsyncDIMM also achieves the highest bandwidth utilization on the NMA side, showing average improvements of $2.33\times$, $1.64\times$, $1.11\times$ and $1.13\times$ over the four baselines. It indicates that, the interference between host execution and NMA execution is minimized in AsyncDIMM, which benefits from the switch method that maximizes row locality.

Second, compared with CK, BK's good locality makes better use of internal bandwidth with FIFO-based NMA MC. However, this locality advantage can be compromised under high host access intensity without scheduling method (e.g., High-1 and High-2). In such case, Chopim and HMC-Link's performance approaches that of synchronous execution, as host and NMA execution constrains each other which diminishes potential performance gains. In contrast, AsyncDIMM demonstrates greater robustness to increased host access intensity.

D. MC Performance

Next, we compare the performance of AsyncDIMM MC with four baselines, using two key MC design metrics: latency and bandwidth. In addition to three asynchronous NMP baselines, we further simulate two architectures for better illustration: 1) IDEAL: An NMP design with a global MC shared by both host and NMAs, which is hardware unrealistic; 2) Host-only: A non-NMP design without NMA execution and NMA access impact. For the former, We verify AsyncDIMM's effectiveness in reducing latency and maximizing bandwidth. For the latter, we examine the impact of NMA execution on host memory access when asynchronous execution is enabled.

1) Latency: Figure 10 illustrates the host access latency (main axis) of AsyncDIMM and other architectures. First, across various host-NMA workload combinations, AsyncDIMM demonstrates lower access latency with decrease of 22%, 47%, 38% compared with UPMEM, Chopim, and HMC-Link, respectively. UPMEM outperforms the other two baselines on workloads Low-1 to Mix-4, but its shielding access fails to exploit rank parallelism, causing a sharply increased latency on workloads High-1 and High-2. For Chopim, the opportunistic access issue often blocks or interferes host access, resulting in higher latency. For HMC-Link, the less efficient NMA MC scheduler limits the latency reduction of host access.

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

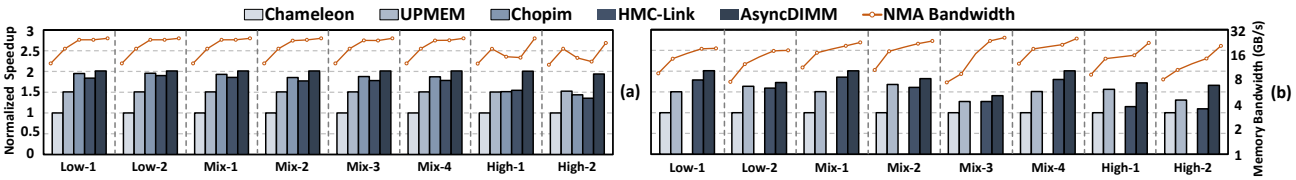


Fig. 9. Overall performance and NMA bandwidth comparison. (a) NMA workloads: BK. (b) NMA workloads: CK.

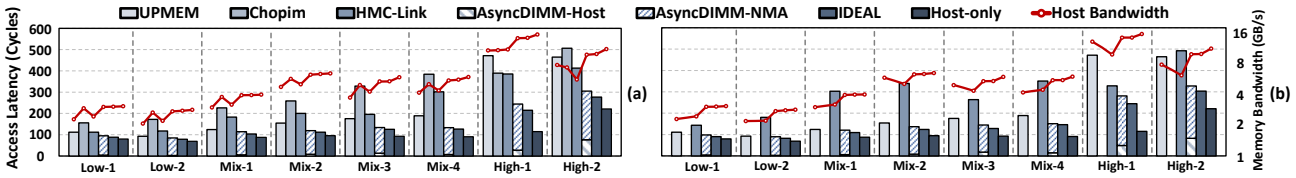


Fig. 10. Host memory access latency and bandwidth comparison. (a) NMA workloads: BK. (b) NMA workloads: CK.

TABLE V
AREA AND POWER OVERHEAD

	Conventional Host MC	AsyncDIMM Host MC	Conventional NMA MC	AsyncDIMM NMA MC	Typical DRAM Chip
Area(mm^2)	2.991	3.103	0.662	0.713	100
Power(W)	0.491	0.613	0.045	0.079	10

Second, compared with IDEAL, AsyncDIMM only increases 11% host access latency on average. It indicates that the OSR mechanism in AsyncDIMM is efficient on offload, schedule, and return, achieving performance close to an ideal, hardware-unrealistic global MC design. Compared with Host-only, AsyncDIMM increases 21%, 38% and 77% latency on host workloads Low, Mix and High on average respectively. This suggests that asynchronous execution is more suitable for workloads Low and Mix, when high real-time performance is required, rather than for workloads High.

2) Bandwidth: Figure 10 also illustrates the host bandwidth (secondary axis). First, AsyncDIMM achieves the highest host bandwidth across all workload combinations, outperforming UPMEM, Chopim, and HMC-Link by factors of 1.34×, 1.15× and 1.32×, respectively. Due to only one rank per channel, UPMEM host cannot exploit the parallelism and is limited to lower bandwidth. The packet-based access in HMC-Link transfers additional control information, which degrades the effective host bandwidth. Both HMC-Link and Chopim achieve higher bandwidth on host workload Low and Mix, but perform less well on workload High.

Second, AsyncDIMM achieves the bandwidth close to IDEAL, and even to Host-only across all Host-NMA workload combinations. The bandwidth loss is less than 3% in most cases and only 12% in the severe cases. It indicates minimal impact from NMA execution on the bandwidth utilization of host access. Consequently, for host workloads demanding stable bandwidth utilization, AsyncDIMM can exploit both host and NMA ability effectively.

E. Overhead

1) MC Design: Table V presents the consumption of area and power under 28nm CMOS technology synthesis. For host MC, AsyncDIMM introduces only 3% area and 24% power overhead, which would be even less when building upon a complex commercial MC. For NMA MC, AsyncDIMM rebuilds the mixed FIFOs, which incurs extra power and area consumption, but it is still acceptable since the DIMM can have a buffer chip of $100mm^2$ and an entire power of $10W$. Compared with Chopim and HMC-Link, AsyncDIMM introduces less hardware overhead as described in SectionII-C.

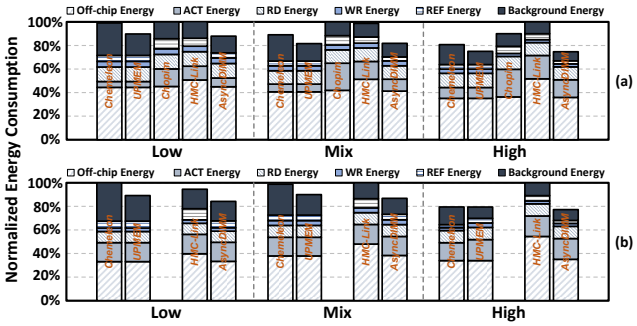


Fig. 11. Normalized energy consumption of memory access comparison. (a) NMA workloads: BK. (b) NMA workloads: CK.

2) Memory Access: Figure 11 compares the energy consumption of memory accesses. We average the results of host workloads Low, Mix, and High respectively for better illustration, as their performance is similar. Compared to Chameleon, Chopim and HMC-Link, AsyncDIMM can achieve 10%, 16%, and 18% energy consumption reduction on average. Compared with UPMEM, AsyncDIMM shows similar energy consumption (slightly lower). Considering that MC designs do not reduce the actual number of memory accesses, the energy consumption mainly comes from higher row locality exploitation (less ACT Energy) and less total execution time (less Background Energy). For example, the ACT energy in UPMEM is around 30% of Chopim and HMC-Link, which saves the total energy consumption even though its total execution time is larger. AsyncDIMM not only exploits row locality but also reduces the total execution time, thus achieving the most significant energy reduction.

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

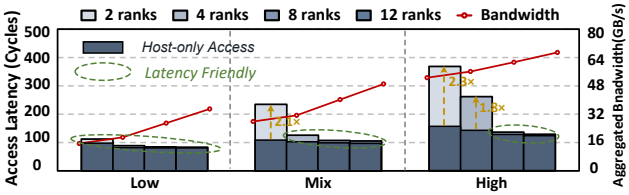


Fig. 12. Scalability comparison with NMA workload BK.

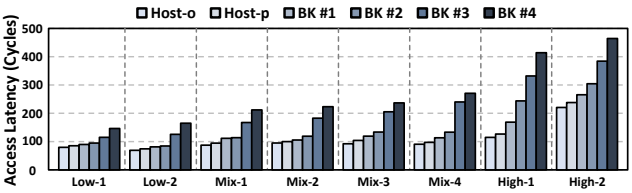


Fig. 13. NMA access intensity comparison with NMA workload BK.

F. Sensitivity Analysis

1) Scalability: We compare AsyncDIMM performance under configurations of different rank numbers to assess its scalability. As shown in Figure 12, when there are two ranks in the channel, only workloads Low show good latency since there is not enough idle time for NMA access, which leads to severe interference between host and NMA accesses. As the number of ranks increases, the memory access latency gradually decreases, and even host workloads High achieve acceptable latency at configurations of 8 and 12 ranks.

2) NMA Access Intensity : We further evaluate the impact of different NMA access intensities on host access latency, as shown in Figure 13. The intensities increase with the BK index (about $1.5\times$ each step and BK#2 is used in previous evaluations). Host-o and Host-p are for host-only (no NMA access) and host-prioritized baselines, respectively. Host-p behaves similarly across BKs, so we simply average it. When the NMA access intensity increases, so does the host memory access latency. Due to the switch method in NMA, the latency does not increase sharply, which avoids the starvation. Meanwhile, Host-p is effective in reducing introduced host latency (only 6%-11% more than Host-o), but at the cost of NMA performance degradation (up to 64% slower, not shown).

3) NMA MC Design: We further consider the high-performance NMA MC (HMC-Link-p) that schedules memory requests with row locality instead of leveraging FIFOs (HMC-Link-f), which HMC-Link can benefit from. As shown in Figure 14, HMC-Link-p shows significant latency reduction of 16%-62%, compared to HMC-Link-f. With NMA workload CK, it shows shorter latency (about 5%-10%) over AsyncDIMM with FIFO-based NMA MC, since it can better mine the row locality. As for the workload BK with regular memory pattern, HMC-Link-p and AsyncDIMM achieve similar performance. Since HMC-Link transfers more data in packets than AsyncDIMM, its performance slightly drops when host access intensity increases. Meanwhile, the high-performance NMA is also compatible with the OSR mechanism, with less hardware cost compared with HMC-Link-p.

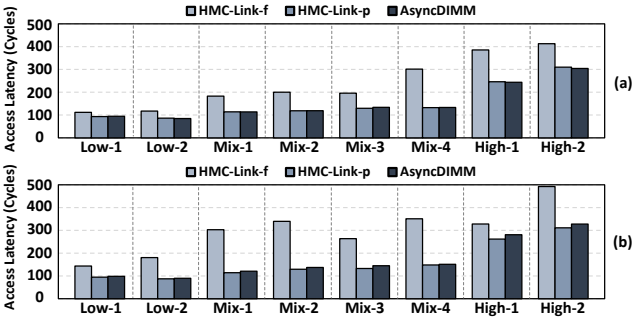


Fig. 14. NMA MC comparison for host access latency. (a) NMA workloads: BK. (b) NMA workloads: CK.

VII. Related Work

On the one hand, during the past couple of decades, continuous research on MC design has been proposed for single-core/single-thread architectures [43], [75], [76], multi-core/multi-thread architectures [45], [77], heterogeneous host architectures [46], [47], heterogeneous storage architectures [63], [78] and so on. Meanwhile, MC designs [79]–[85] modify the buffer chip or memory buses to improve bus bandwidth or reduce power consumption. However, the distributed MCs in DIMM-NMP pose new challenges to MC design.

On the other hand, many DIMM-NMP accelerators have been proposed for recommendation systems [4]–[9], genomics analysis [12], [13], DNN acceleration [10], [11], sparse tensor/vector algebra [31], [86]. Interconnections [33], [54], [87] are introduced for inter-DIMM/bank communication. Works [53], [56] implement new communication mechanisms, while works [5], [6], [10], [64] introduce new instructions for NMA execution. However, DIMM-NMP architectures lack an efficient asynchronous execution method. Though NDA [23], UPMEM [2], Chopim [16] contribute valuable ideas for it, they are restricted in terms of generality and performance.

VIII. Conclusion

In this paper, we propose AsyncDIMM, a novel DIMM-NMP design with asynchronous execution based on existing memory buses. It enables single access mode (host or NMA), concurrent access mode, and a seamless switch between them. We build an efficient offload-schedule-return mechanism with explicit/implicit synchronization between MCs at the host-side and the NMA-side. We present a detailed design with limited hardware modifications to conventional host and NMA MCs, which is extensively validated on the FPGA. Comprehensive experiments demonstrate that AsyncDIMM outperforms four NMP baselines by $1.19\times$-$1.92\times$ on average, enabling efficient asynchronous execution with up to $2.25\times$ bandwidth utilization uplift and 47% access latency reduction.

IX. Acknowledgement

We sincerely thank anonymous reviewers for their insightful suggestions. This work was supported by National Natural Science Foundation of China (Grant No.92264108 and U24B20165). Naifeng Jing is the corresponding author.

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] H. Asghari-Moghaddam, Y. H. Son, J. H. Ahn, and N. S. Kim, "Chameleon: Versatile and practical near-dram acceleration architecture for large memory systems," in *2016 49th annual IEEE/ACM international symposium on Microarchitecture (MICRO)*. IEEE, 2016, pp. 1–13.

[2] F. Devaux, "The true processing in memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, 2019, pp. 1–24.

[3] L. Ke, X. Zhang, J. So, J.-G. Lee, S.-H. Kang, S. Lee, S. Han, Y. Cho, J. H. Kim, Y. Kwon *et al.*, "Near-memory processing in action: Accelerating personalized recommendation with axdimm," *IEEE Micro*, vol. 42, no. 1, pp. 116–127, 2021.

[4] Y. Kwon, Y. Lee, and M. Rhu, "Tensordimm: A practical near-memory processing architecture for embeddings and tensor operations in deep learning," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 740–753.

[5] J. Park, B. Kim, S. Yun, E. Lee, M. Rhu, and J. H. Ahn, "Trim: Enhancing processor-memory interfaces with scalable tensor reduction in memory," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 268–281.

[6] L. Ke, U. Gupta, B. Y. Cho, D. Brooks, V. Chandra, U. Diril, A. Firoozshahian, K. Hazelwood, B. Jia, H.-H. S. Lee *et al.*, "Recnmp: Accelerating personalized recommendation with near-memory processing," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 790–803.

[7] B. Asgari, R. Hadidi, J. Cao, S.-K. Lim, H. Kim *et al.*, "Fafnir: Accelerating sparse gathering by using efficient near-memory intelligent reduction," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 908–920.

[8] C. Li, Z. Zhou, X. Li, G. Sun, and D. Niu, "Nmexplorer: An efficient exploration framework for dimm-based near-memory tensor reduction," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[9] H. Liu, L. Zheng, Y. Huang, C. Liu, X. Ye, J. Yuan, X. Liao, H. Jin, and J. Xue, "Accelerating personalized recommendation with cross-level near-memory processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[10] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "Gradpim: A practical processing-in-dram architecture for gradient descent," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 249–262.

[11] L. Liu, J. Lin, Z. Qu, Y. Ding, and Y. Xie, "Enmc: Extreme near-memory classification via approximate screening," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1309–1322.

[12] W. Huangfu, X. Li, S. Li, X. Hu, P. Gu, and Y. Xie, "Medal: Scalable dimm based near data processing accelerator for dna seeding algorithm," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 587–599.

[13] W. Huangfu, K. T. Malladi, A. Chang, and Y. Xie, "Beacon: Scalable near-data-processing accelerators for genome analysis near memory pool with the cxl support," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 727–743.

[14] D. Chen, H. He, H. Jin, L. Zheng, Y. Huang, X. Shen, and X. Liao, "Metanmp: Leveraging cartesian-like product to accelerate hgnns with near-memory processing," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–13.

[15] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a new paradigm: Experimental analysis and characterization of a real processing-in-memory system," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.

[16] B. Y. Cho, Y. Kwon, S. Lym, and M. Erez, "Near data acceleration with concurrent host access," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 818–831.

[17] "JEDEC Standard: DDR4 SDRAM Load Reduced DIMM (LRDIMM) Design Specification." JEDEC,Standard, 2014.

[18] A. Devic, S. B. Rai, A. Sivasubramaniam, A. Akel, S. Eilert, and J. Eno, "To pim or not for emerging general purpose processing in ddr memory systems," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 231–244.

[19] D. Chen, H. Jin, L. Zheng, Y. Huang, P. Yao, C. Gui, Q. Wang, H. Liu, H. He, X. Liao *et al.*, "A general offloading approach for near-dram processing-in-memory architectures," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2022, pp. 246–257.

[20] M. Alian, S. W. Min, H. Asgharimoghaddam, A. Dhar, D. K. Wang, T. Roewer, A. McPadden, O. O'Halloran, D. Chen, J. Xiong *et al.*, "Application-transparent near-memory processing architecture with memory channel network," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 802–814.

[21] J. L. Henning, "Spec cpu2006 benchmark descriptions," *ACM SIGARCH Computer Architecture News*, vol. 34, no. 4, pp. 1–17, 2006.

[22] "SPEC CPU2017," [Online], Standard Performance Evaluation Corporation, 201, avalable: https://www.spec.org/cpu2017.

[23] A. Farmahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, "Nda: Near-dram acceleration architecture leveraging commodity dram devices and standard memory modules," in *2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2015, pp. 283–295.

[24] J. T. Pawlowski, "Hybrid memory cube (hmc)," in *2011 IEEE Hot chips 23 symposium (HCS)*. IEEE, 2011, pp. 1–24.

[25] J. Jeddeloh and B. Keeth, "Hybrid memory cube new dram architecture increases density and performance," in *2012 symposium on VLSI technology (VLSIT)*. IEEE, 2012, pp. 87–88.

[26] M. Radulović, "Memory bandwidth and latency in hpc: system requirements and performance impact," 2019.

[27] S. Ghose, T. Li, N. Hajinazar, D. S. Cali, and O. Mutlu, "Demystifying complex workload-dram interactions: An experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 3, no. 3, pp. 1–50, 2019.

[28] M. Gupta, D. Roberts, M. Meswani, V. Sridharan, D. Tullsen, and R. Gupta, "Reliability and performance trade-off study of heterogeneous memories," in *Proceedings of the Second International Symposium on Memory Systems*, 2016, pp. 395–401.

[29] H. Kal, C. Yoo, and W. W. Ro, "Aespa: Asynchronous execution scheme to exploit bank-level parallelism of processing-in-memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 815–827.

[30] Y. K. Zhao, S. Wu, J. Zhang, S. Li, C. Li, and Y. C. Lin, "Instant-nerf: Instant on-device neural radiance field training via algorithm-accelerator co-designed near-memory processing," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.

[31] S. Feng, X. He, K.-Y. Chen, L. Ke, X. Zhang, D. Blaauw, T. Mudge, and R. Dreslinski, "Menda: A near-memory multi-way merge solution for sparse transposition and dataflows," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 245–258.

[32] A. Boroumand, S. Ghose, M. Patel, H. Hassan, B. Lucia, R. Ausavarungnirun, K. Hsieh, N. Hajinazar, K. T. Malladi, H. Zheng *et al.*, "Conda: Efficient cache coherence support for near-data accelerators," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 629–642.

[33] J. Cong, Z. Fang, M. Gill, F. Javadi, and G. Reinman, "Aim: accelerating computational genomics through scalable and noninvasive accelerator-interposed memory," in *Proceedings of the International Symposium on Memory Systems*, 2017, pp. 3–14.

[34] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah *et al.*, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–26.

[35] Z. Zhou, C. Li, X. Wei, X. Wang, and G. Sun, "Gnnear: Accelerating full-batch training of graph neural networks with near-memory processing," in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*, 2022, pp. 54–68.

[36] Y. Kwon, K. Vladimir, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An *et al.*, "System architecture and software stack for gddr6-aim," in *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE, 2022, pp. 1–25.

[37] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, "Newton: A dram-maker's accelerator-in-memory (aim) architecture for machine learning," in *2020 53rd Annual IEEE/ACM*

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

International Symposium on Microarchitecture (MICRO). IEEE, 2020, pp. 372–385.

[38] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 2015, pp. 105–117.

[39] D. D. Sharma, "Compute express link," *CXL Consortium White Paper*, 2019.

[40] "DDR4 SDRAM LRDIMM," [Online], Micron Technology, 2015, avalable: https://www.micron.com/content/dam/micron/global/secure/spectek/data-sheet/dram/ddr4/spectek-8gb-ddr4-sdram.pdf.

[41] R. Balasubramonian, *Innovations in the memory system*. Springer Nature, 2022.

[42] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, "A case for exploiting subarray-level parallelism (salp) in dram," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 3, pp. 368–379, 2012.

[43] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," *ACM SIGARCH Computer Architecture News*, vol. 28, no. 2, pp. 128–138, 2000.

[44] L. Gopalakrishnan, V. Thyagarajan, P. Kole, and G. R. Gangula, "Memory controller with reconfigurable hardware," Mar. 24 2015, uS Patent 8,990,490.

[45] O. Mutlu and T. Moscibroda, "Parallelism-aware batch scheduling: Enhancing both performance and fairness of shared dram systems," *ACM SIGARCH Computer Architecture News*, vol. 36, no. 3, pp. 63–74, 2008.

[46] R. Ausavarungnirun, K. K.-W. Chang, L. Subramanian, G. H. Loh, and O. Mutlu, "Staged memory scheduling: Achieving high performance and scalability in heterogeneous systems," *ACM SIGARCH Computer Architecture News*, vol. 40, no. 3, pp. 416–427, 2012.

[47] L. Subramanian, V. Seshadri, Y. Kim, B. Jaiyen, and O. Mutlu, "Mise: Providing performance predictability and improving fairness in shared main memory systems," in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2013, pp. 639–650.

[48] UPMEM, "Upmem sdk version 2024.2.0," 2024. [Online]. Available: https://sdk.upmem.com/2024.2.0/

[49] Y. Zhuo, C. Wang, M. Zhang, R. Wang, D. Niu, Y. Wang, and X. Qian, "Graphq: Scalable pim-based graph processing," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 712–725.

[50] M. Zhang, Y. Zhuo, C. Wang, M. Gao, Y. Wu, K. Chen, C. Kozyrakis, and X. Qian, "Graphp: Reducing communication for pim-based graph processing with efficient data partition," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 544–557.

[51] "Hybrid Memory Cube - HMC Gen2," [Online], Micron Technology, 2017, avalable: https://www.mouser.com/datasheet/2/671/hmc_gen2-1382047.pdf.

[52] R. Hadidi, B. Asgari, B. A. Mudassar, S. Mukhopadhyay, S. Yalamanchili, and H. Kim, "Demystifying the characteristics of 3d-stacked memories: A case study for hybrid memory cube," in *2017 IEEE international symposium on Workload characterization (IISWC)*. IEEE, 2017, pp. 66–75.

[53] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, "Abc-dimm: Alleviating the bottleneck of communication in dimm-based near-memory processing with inter-dimm broadcast," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 237–250.

[54] Z. Zhou, C. Li, F. Yang, and G. Sun, "Dimm-link: Enabling efficient inter-dimm communication for near-memory processing," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 302–316.

[55] "Designware enhanced universal ddr memory controller," [Online], Synopsis, 2018, avalable: https://www.synopsys.com/dw/doc.php/ds/c/dwc_ddr_csds.pdf.

[56] L. Chen, J. Jiang, Q. Wang, Z. Mao, and N. Jing, "Bridge-ndp: Achieving efficient communication-computation overlap in near data processing with bridge architecture," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 460–465.

[57] A. Cosoroaba, "Memory interfaces made easy with xilinx fpgas and the memory interface generator," *Xilinx Corporation, white paper*, vol. 260, 2007.

[58] B. Ananth, T. Niraj, V. Sriram, and D. Shivank, "Ddr4memorycontroller," 2017. [Online]. Available: https://github.com/ananthbhat94/DDR4MemoryController

[59] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 722–737.

[60] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 103–119.

[61] R. Panda, S. Song, J. Dean, and L. K. John, "Wait of a decade: Did spec cpu 2017 broaden the performance horizon?" in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 271–282.

[62] M. M. Rafique and Z. Zhu, "Faps-3d: Feedback-directed adaptive page management scheme for 3d-stacked dram," in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 373–382.

[63] T. Le, Z. Zhang, and Z. Zhu, "Pomi: Polling-based memory interface for hybrid memory system," in *2021 IEEE 39th International Conference on Computer Design (ICCD)*. IEEE, 2021, pp. 447–455.

[64] Y. Zhao, M. Gao, F. Liu, Y. Hu, Z. Wang, H. Lin, J. Li, H. Xian, H. Dong, T. Yang *et al.*, "Um-pim: Dram-based pim with uniform & shared memory space," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 644–659.

[65] F. Zeng, L. Qiao, M. Liu, and Z. Tang, "Memory performance characterization of spec cpu2006 benchmarks using tsim," *Physics Procedia*, vol. 33, pp. 1029–1035, 2012.

[66] A. Nag and R. Balasubramonian, "Orderlight: Lightweight memory-ordering primitive for efficient fine-grained pim computations," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 298–310.

[67] J. D. McCalpin *et al.*, "Memory bandwidth and machine balance in current high performance computers," *IEEE computer society technical committee on computer architecture (TCCA) newsletter*, vol. 2, no. 19-25, 1995.

[68] L. S. Blackford, A. Petitet, R. Pozo, K. Remington, R. C. Whaley, J. Demmel, J. Dongarra, I. Duff, S. Hammarling, G. Henry *et al.*, "An updated set of basic linear algebra subprograms (blas)," *ACM Transactions on Mathematical Software*, vol. 28, no. 2, pp. 135–151, 2002.

[69] P. Duhamel and M. Vetterli, "Fast fourier transforms: a tutorial review and a state of the art," *Signal processing*, vol. 19, no. 4, pp. 259–299, 1990.

[70] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH computer architecture news*, vol. 39, no. 2, pp. 1–7, 2011.

[71] J. Lowe-Power, A. M. Ahmad, A. Akram, M. Alian, R. Amslinger, M. Andreozzi, A. Armejach, N. Asmussen, B. Beckmann, S. Bharadwaj *et al.*, "The gem5 simulator: Version 20.0+," *arXiv preprint arXiv:2007.03152*, 2020.

[72] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[73] N. Muralimanohar, R. Balasubramonian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.

[74] N. P. Jouppi, A. B. Kahng, N. Muralimanohar, and V. Srinivas, "Cacti-io: Cacti with off-chip power-area-timing models," in *Proceedings of the International Conference on Computer-Aided Design*, 2012, pp. 294–301.

[75] Z. Zhang, Z. Zhu, and X. Zhang, "A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality," in *Proceedings of the 33rd annual ACM/IEEE international symposium on Microarchitecture*, 2000, pp. 32–41.

[76] I. Hur and C. Lin, "Adaptive history-based memory schedulers," in *37th International Symposium on Microarchitecture (MICRO-37'04)*. IEEE, 2004, pp. 343–354.

[77] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, "Thread cluster memory scheduling: Exploiting differences in memory access

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.

behavior," in *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2010, pp. 65–76.

[78] S. Song, A. Das, O. Mutlu, and N. Kandasamy, "Aging-aware request scheduling for non-volatile main memory," in *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021, pp. 657–664.

[79] Z. Cui, T. Lu, S. A. McKee, M. Chen, H. Pan, and Y. Ruan, "Twin-load: Bridging the gap between conventional direct-attached and buffer-on-board memory systems," in *Proceedings of the Second International Symposium on Memory Systems*, 2016, pp. 164–176.

[80] K. Fang, L. Chen, Z. Zhang, and Z. Zhu, "Memory architecture for integrating emerging memory technologies," in *2011 International Conference on Parallel Architectures and Compilation Techniques*. IEEE, 2011, pp. 403–412.

[81] J. Haas and P. Vogt, "Fully-buffered dimm technology moves enterprise platforms to the next level," *Technology@ Intel Magazine*, vol. 3, 2005.

[82] H. Zheng, J. Lin, Z. Zhang, E. Gorbatov, H. David, and Z. Zhu, "Mini-rank: Adaptive dram architecture for improving memory power efficiency," in *2008 41st IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2008, pp. 210–221.

[83] H. Zheng, J. Lin, Z. Zhang, and Z. Zhu, "Decoupled dimm: building high-bandwidth memory system using low-speed dram devices," *ACM SIGARCH Computer Architecture News*, vol. 37, no. 3, pp. 255–266, 2009.

[84] "Intel optane data center persistent memory," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, 2019, pp. i–xxv.

[85] J. Izraelevitz, J. Yang, L. Zhang, J. Kim, X. Liu, A. Memaripour, Y. J. Soh, Z. Wang, Y. Xu, S. R. Dulloor *et al.*, "Basic performance measurements of the intel optane dc persistent memory module," *arXiv preprint arXiv:1903.05714*, 2019.

[86] B. Y. Cho, J. Jung, and M. Erez, "Accelerating bandwidth-bound deep learning inference with main-memory accelerators," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.

[87] B. Tian, Y. Li, L. Jiang, S. Cai, and M. Gao, "Ndpbridge: Enabling cross-bank coordination in near-dram-bank processing architectures," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 628–643.

Authorized licensed use limited to: Korea Electronics Technology Institute. Downloaded on April 17,2025 at 08:21:29 UTC from IEEE Xplore. Restrictions apply.